More String Methods

Concepts

- Classification Methods
  - isalpha()
  - isdecimal()
  - islower()
  - isupper()
  - isalnum()
- Case Conversion Methods
  - capitalize()
  - title()
  - swapcase()
- Counting and Searching Methods
  - count()
  - index()
  - rindex()
  - find()
  - rfind()

1. Classification Methods

These methods are used to check the characteristics of individual characters in a string.

1.1 Isalpha

Syntax: `str_var.isalpha()`

Gives `True` if all the characters are alphabets. Otherwise, `False`

Example 1:

Code

```python
is_alpha = "Rahul".isalpha()
print(is_alpha)
```

Output

Python 3.10

DEBUG RUN CODE

Your Input

Your Input

Your Input

Your Input

```
True
```

Example 2:

Code

```python
is_alpha = "Rahul@123".isalpha()
print(is_alpha)
```

Output

```
False
```

1.2 Isdecimal

Syntax: `str_var.isdecimal()`

Gives `True` if all the characters are decimals.
Otherwise, `False`

Example 1:

Code

```python
is_decimal = "9876543210".isdecimal()
print(is_decimal)
```

Output

```
True
```

Example 2:

Code

```python
is_decimal = "123.R".isdecimal()
print(is_decimal)
```

Output

```
False
```

1.3 Islower

Syntax: `str_var.islower()`

Gives `True` if all letters in the string are in lowercase.
Otherwise, `False`

Example 1:

Code

```python
is_lower = "hello ravi!".islower()
print(is_lower)
```

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

Output

```
True
```

Example 2:

Code

PYTHON

```
is_lower = "Hello Ravi!".islower()
print(is_lower)
```

Output

```
False
```

1.4 Isupper

Syntax: `str_var.isupper()`

Gives `True` if all letters in the string are in uppercase. Otherwise, `False`

Example 1:

Code

PYTHON

```
is_upper = "HELLO RAVI!".isupper()
print(is_upper)
```

Output

```
True
```

Example 2:

Code

PYTHON

```
is_upper = "hELLO rAVI!".isupper()
print(is_upper)
```

Output

```
False
```

1.5 Isalnum

Syntax: `str_var.isalnum()`

Gives `True` if the string is alphanumeric (a letter or a number). Otherwise, `False`

Example 1:

Code

PYTHON

```
is_alnum = "Rahul123".isalnum()
```

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

```
print(is_alnum)
```

Output

```
True
```

Example 2:

Code

PYTHON

```
is_alnum = "Rahul".isalnum()
print(is_alnum)
```

Output

```
True
```

Example 3:

Code

PYTHON

```
is_alnum = "Rahul@123".isalnum()
print(is_alnum)
```

Output

```
False
```

2. Case Conversion Methods

These methods are used to change the case of a string.

2.1 Capitalize

Syntax: `str_var.capitalize()`

Gives a new string after converting the first letter in the string to uppercase and all other letters to lowercase.

Code

PYTHON

```
capitalized = "the Planet Earth".capitali
print(capitalized)
```

Output

```
The planet earth
```

2.2 Title

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

Syntax: `str_var.title()`

Gives a new string after converting the first letter of every word to uppercase.

If a word contains a number or a special character, the first letter after that is converted to uppercase.

Example 1:

Code

PYTHON

```python
title_case = "the planet earth".title()
print(title_case)
```

Output

```
The Planet Earth
```

Example 2:

Code

PYTHON

```python
title_case = "my_name#is john1doe and i l
print(title_case)
```

Output

```
My_Name#Is John1Doe And I Love Python
```

2.3 Swapcase

Syntax: `str_var.swapcase()`

Gives a new string after converting the uppercase letters to lowercase and vice-versa.

Code

PYTHON

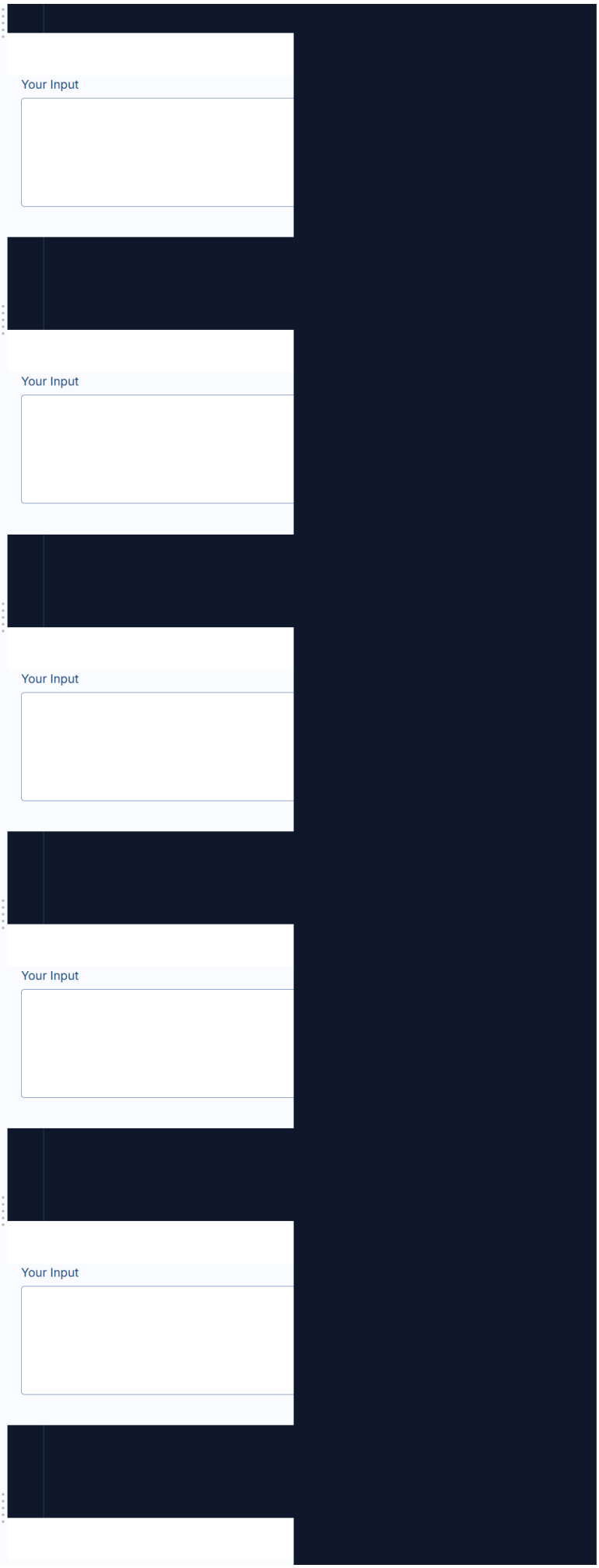
```python
swapped = "mY nAME IS rAVI".swapcase()
print(swapped)
```

Output

```
My Name is Ravi
```

3. Counting and Searching Methods

These methods are used to count the occurrences of a substring in a string and to find the position of a substring



in a string.

3.1 Count

Syntax: `str_var.count(str, start_index, end_index)`

Here, the `start_index` and the `end_index` are optional.

The `count()` method gives the number of times the specified string `str` appears in the string. It searches the complete string as default.

If `start_index` and `end_index` are provided, it searches between these indices. The `end_index` is not included.

Example 1:

Code

PYTHON

```
text = "Hello, world!"
letter_count = text.count("l")
print(letter_count)
```

Output

```
3
```

Example 2:

Code

PYTHON

```
text = "Hello, world!"
letter_count = text.count("l", 2, 10)
print(letter_count)
```

Output

```
2
```

3.2 Index

Syntax: `str_var.index(str, start_index, end_index)`

Here, the `start_index` and the `end_index` are optional.

The `index()` method gives the index at the first occurrence of the specified string `str` .

It results in an error if the specified string `str` is not found.

The `index()` method searches the complete string as default. If `start_index` and `end_index` are provided, it searches between these indices. The `end_index` is not included.

Example 1:

Code

PYTHON

```
text = "I have a spare key, if I lose my
word_index = text.index("key")
print(word_index)
```

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

Output

```
15
```

Example 2:

Code

PYTHON

```
text = "coo coo"
word_index = text.index("co", 3, 6)
print(word_index)
```

Output

```
4
```

Example 3:

Code

PYTHON

```
text = "coo coo"
word_index = text.index("ha")
print(word_index)
```

Output

```
ValueError: substring not found
```

3.3 rIndex

Syntax: `str_var.rindex(str, start_index, end_index)`

Here, the `start_index` and the `end_index` are optional.

The `rindex()` method gives the index at the last occurrence of the specified string `str` .

It results in an error if the specified string `str` is not found.

The `rindex()` method searches the complete string as default. If `start_index` and `end_index` are provided, it searches between these indices. The `end_index` is not included.

Example 1:

Code

PYTHON

```
text = "I have a spare key, if I lose my
word_index = text.rindex("key")
print(word_index)
```

Output

```
33
```

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

Example 2:

Code

```python
text = "coo coo coo"
word_index = text.rindex("co", 3, 10)
print(word_index)
```

Output

```
8
```

Example 3:

Code

```python
text = "coo coo"
word_index = text.rindex("ha")
print(word_index)
```

Output

```
ValueError: substring not found
```

3.4 Find

Syntax: `str_var.find(str, start_index, end_index)`

Here, the `start_index` and the `end_index` are optional.

The `find()` method gives the index at the first occurrence of the specified string `str` .

If the specified string `str` is not found, it returns `-1` .

The `find()` method searches the complete string as default. If `start_index` and `end_index` are provided, it searches between these indices. The `end_index` is not included.

It works similarly to the `index()` method. The only difference is that the `index()` method results in an error if the specified string is not found, while `find()` does not.

Example 1:

Code

```python
text = "I have a spare key, if I lose my
word_index = text.find("key")
print(word_index)
```

Output

```
15
```

Example 2:

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

Code

```python
text = "coo coo"
word_index = text.find("co", 3, 6)
print(word_index)
```

Output

```
4
```

Example 3:

Code

```python
text = "coo coo"
word_index = text.find("ha")
print(word_index)
```

Output

```
-1
```

3.5 rFind

Syntax: `str_var.rfind(str, start_index, end_index)`

Here, the `start_index` and the `end_index` are optional.

The `rfind()` method gives the index at the last occurrence of the specified string `str` .

If the specified string `str` is not found, it returns `-1` .

The `rfind()` method searches the complete string as default. If `start_index` and `end_index` are provided, it searches between these indices. The `end_index` is not included.

It works similarly to the `rindex()` method. The only difference is that the `rindex()` method results in an error if the specified string is not found, while `rfind()` does not.

Example 1:

Code

```python
text = "I have a spare key, if I lose my
word_index = text.rfind("key")
print(word_index)
```

Output

```
33
```

Example 2:

Code

```python
```

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

```
text = "coo coo coo"
```

Your input